First we begin by parsing the arguments, and store the addresses from the text file in an array. We then store their associated page numbers in a separate array that we use when running optimal or LRU.
We decided to make both our TLB and page table both lists of tuples, because Python allows us to treat lists like queues. This was helpful for FIFO.

For every address, we calculate the page number we bit shift to the right by 8, and to get the offset we & it by 0xFF. We check if it exists in the TLB, and if it does that counts as a TLB hit. Then we check if it exists in the TLB, and if it doesn't exist there but exists in the PAGE_TABLE, that counts as a TLB miss. We traverse through our page table to find the associated frame number of the page number, and update the TLB. If the page number of the address doesn't exist in the TLB or the page table, then that counts as a TLB miss and a page table miss. In this case, we check if there are any available frames left.If there are then we assign the available frame number to the page number and append the tuple to the page table and TLB. If there are no available frames, we invoke the appropriate algorithm. Any time we want to add a tuple to the TLB we first check if it is full, and if it is then we use FIFO.

The FIFO algorithm for the page table is pretty simple. We pop from the page table and assign the frame number from that tuple to the new page number. We append this new tuple to the page table and update the TLB.

If it's the case that the user wants to use the LRU algorithm, then the program will iterate through all pages in memory and use the CURRENT_INDEX variable to move backward to index 0, searching through the SEQ array to find the most recent occurrence of the current page. The least_recent_use and last_use variables are compared against each other to keep track of the index of the page that was least recently used.

If it's the case that the user wants to use the optimal algorithm, then like the LRU algorithm, instead of the least_recent_use variable, the program uses the farthest-use variable to track the maximum distance into the future before a page will be needed again. A loop is used to scan forward from the current index until the end of the SEQ array, searching for the next occurrence of the current page. If a page is not found in any future reference, then the algorithm selects the current page in the iteration as the victim to be replaced. If all pages will be used again, then their next use positions are compared and the page with the largest next_use index becomes the victim of replacement.
Page fault rate is calculated by dividing the number of page table misses by the number of addresses translated. TLB hit rate is calculated by dividing the number of TLB hits by the number of addresses translated.